



Agentic AI Task

AI Governance — Problem Statements

June 2026

Prepared by: Aivar Innovations

Introduction

Your Task

Pick any one problem statement and implement a production-ready solution. You are expected to deliver a working system — not a prototype, not a slide deck, not a demo script.

Production Readiness Matters

Solutions will be evaluated on production readiness. A working solution that runs only on localhost will score lower than one that is deployed and governs real AI workloads in a cloud environment. The closer your solution is to something an enterprise could actually adopt, the higher it will be rated.

The following are examples of what production readiness means in practice:

- A solution deployed on AWS (Lambda, ECS, EKS, or equivalent) that can govern agents also hosted on AWS scores higher than one running on localhost.
- A solution that handles concurrent requests, persists state, and exposes a usable API scores higher than one that runs as a single script.
- A solution with logging, error handling, and a basic health check scores higher than one without.
- A solution that connects to at least one real LLM provider (OpenAI, Anthropic, AWS Bedrock, etc.) scores higher than one using mocked responses only.

Extra points are awarded for deployment quality, integration breadth, and the degree to which the solution could be plugged into a real enterprise AI stack without significant rework.

Problem Statement Format

Each problem statement contains the following sections:

- Context — the real-world pain this problem addresses and why current tools fail to solve it.
- The Challenge — one sentence stating exactly what you are building.
- What to Build — the concrete components and capabilities your solution must include.
- Success Criteria — the testable conditions your solution must satisfy to be considered complete.
- Bonus — optional extensions that demonstrate depth and earn extra credit.

Problems are grouped into ten capability units. Each unit represents a distinct layer of the AI governance stack. You may pick any problem from any unit.

Unit 6 Compliance & Regulatory Management

Compliance that lives in a document is not compliance. These problems address the gap between what regulators require and what is actually enforced — with a focus on LLM and agent-specific obligations that existing compliance platforms have not yet solved.

PS-6.1

Agent Compliance Card Generator

Context

EU AI Act Article 13 requires transparency documentation for high-risk AI systems. Model cards exist for ML models. Nothing equivalent exists for agents — a document that captures what an agent does, what tools it can invoke, what data it accesses, what decisions it can influence, what human oversight is in place, and what its risk classification is. Compliance teams are generating this manually in Word documents.

The Challenge

Build an automated agent compliance card generator that produces a structured, regulation-aligned transparency document for any agent from its configuration and runtime data.

What to Build

- An agent card schema covering: purpose and scope, LLM used with version, tool inventory with operations and data access, data sources, decision authority (informational / advisory / autonomous), human oversight mechanisms, risk classification, known limitations, and incident contact.
- A card generator that populates the schema by parsing an agent config file, a tool manifest, and a sample run trace.
- A regulation mapper: annotate each card field with the regulatory obligation it satisfies — EU AI Act Article 13, NIST AI RMF Govern function, ISO 42001 clause.
- A completeness checker: flag any card fields that are missing or contain placeholder values.
- Export as structured JSON and as a human-readable document.

Success Criteria

- Given three agent configs of increasing complexity, three accurate agent cards are generated.
- Regulation mapper correctly annotates all fields with at least one framework reference.
- Completeness checker correctly flags an intentionally incomplete card.

- Human-readable output is clear enough for a non-technical compliance reviewer to understand.

Bonus

- Add a card diff view: when an agent is updated, show which card fields changed and flag any changes that require a regulatory re-assessment.

PS-6.2

Runtime-to-Compliance Bridge

Context

A compliance platform says an agent's guardrails are active and compliant. In production, the guardrail is misconfigured and has been silently failing for two weeks. The compliance system has no way to know because it gets its data from periodic self-assessments, not from live runtime signals. The result is a compliance record that is demonstrably false.

The Challenge

Build a bridge that continuously syncs runtime enforcement state into a compliance record — so compliance status reflects what is actually running, not what was documented at deployment.

What to Build

- A compliance record schema for an agent: `guardrails_active`, `last_violation_timestamp`, `pii_redaction_enabled`, `audit_logging_enabled`, `hitl_configured`, and `overall_compliance_status` (green / amber / red).
- A runtime telemetry collector: poll or receive webhooks from a running agent's guardrail system and audit logger.
- A sync engine: update the compliance record every five minutes based on live telemetry; if a guardrail check that should be active has produced zero events for 24 hours, flag it as potentially inactive.
- A compliance status timeline: show how compliance status changed over the last seven days with the triggering event for each change.

Success Criteria

- Compliance record correctly reflects active guardrails when all systems are functioning.
- A simulated guardrail failure triggers a status change from green to amber within 24 hours and red within 48 hours.
- Timeline correctly shows status transitions and timestamps.